

Yes, you can absolutely do that! The system can track your direct displacement from the start point and figure out if you are moving closer to or farther away from your origin.

To track whether you are moving **"Away"** from the starting point or returning **"Back"** toward it, we use a simple geometry comparison: we check if your new distance from the origin (0,0) is greater or smaller than your previous distance. Here is the updated **MicroPython script for your offline RT1064 EVK board (Thonny)**. It calculates the live distance, checks your approach status, and runs completely offline with no external dependencies.

Updated MicroPython Script (For Thonny)

```
import machine
import time
import math

# 1. Hardware I2C Setup for RT1064 EVK
i2c = machine.I2C(1, baudrate=100000)

# LSM303LDHC I2C Device Addresses
ACCEL_ADDR = 0x19
MAG_ADDR = 0x1E

def init_lsm303():
    i2c.writeto_mem(ACCEL_ADDR, 0x20, b'\x57')
    i2c.writeto_mem(MAG_ADDR, 0x02, b'\x00')

def read_sensor_data():
    accel_raw = i2c.readfrom_mem(ACCEL_ADDR, 0x28 | 0x80, 6)
    ax = int.from_bytes(accel_raw[0:2], 'little')
    ay = int.from_bytes(accel_raw[2:4], 'little')
    az = int.from_bytes(accel_raw[4:6], 'little')

    mag_raw = i2c.readfrom_mem(MAG_ADDR, 0x03, 6)
    mx = int.from_bytes(mag_raw[0:2], 'big')
    mz = int.from_bytes(mag_raw[2:4], 'big')
    my = int.from_bytes(mag_raw[4:6], 'big')

    conv = lambda v: v - 65536 if v > 32767 else v
    return conv(ax), conv(ay), conv(az), conv(mx), conv(my), conv(mz)

def calculate_heading(mx, my):
    heading = math.atan2(my, mx)
    if heading < 0:
        heading += 2 * math.pi
    return heading

def get_compass_direction(deg):
    if (deg >= 337.5) or (deg < 22.5): return "North"
    elif (deg >= 22.5) and (deg < 67.5): return "North-East"
    elif (deg >= 67.5) and (deg < 112.5): return "East"
    elif (deg >= 112.5) and (deg < 157.5): return "South-East"
    elif (deg >= 157.5) and (deg < 202.5): return "South"
    elif (deg >= 202.5) and (deg < 247.5): return "South-West"
    elif (deg >= 247.5) and (deg < 292.5): return "West"
    else: return "North-West"
```

```

def get_movement_turn(current, previous):
    if previous is None: return "Starting Movement"
    diff = current - previous
    while diff > 180: diff -= 360
    while diff < -180: diff += 360
    if abs(diff) < 15: return "Moving Straight"
    elif diff > 0: return "Turning Right"
    else: return "Turning Left"

# --- Navigation & Displacement Variables ---
step_count = 0
x_pos = 0.0
y_pos = 0.0
total_distance = 0.0
STRIDE_LENGTH = 1.0 # Set to 1.0 unit per step for easy math tracking
prev_heading_deg = None
prev_straight_line_dist = 0.0

STEP_THRESHOLD = 4500
GRAVITY_BASELINE = 16000
step_cooldown_timer = 0

def process_step(mx, my):
    global step_count, x_pos, y_pos, total_distance, prev_heading_deg, prev_straight_line_dist

    step_count += 1
    total_distance += STRIDE_LENGTH

    heading_rad = calculate_heading(mx, my)
    heading_deg = math.degrees(heading_rad)

    movement = get_movement_turn(heading_deg, prev_heading_deg)
    compass_dir = get_compass_direction(heading_deg)
    prev_heading_deg = heading_deg

    # Update grid positioning
    x_pos += STRIDE_LENGTH * math.sin(heading_rad)
    y_pos += STRIDE_LENGTH * math.cos(heading_rad)

    # Calculate exact distance from center origin (0,0)
    straight_line_dist = math.sqrt((x_pos ** 2) + (y_pos ** 2))

    # Determine if moving away or coming back to origin center
    if step_count == 1:
        origin_relation = "Moving Away from Origin"
    elif straight_line_dist < prev_straight_line_dist:
        origin_relation = "Moving BACKWARDS / Returning to Origin"
    else:
        origin_relation = "Moving FORWARD / Away from Origin"

    prev_straight_line_dist = straight_line_dist

    # --- Live Terminal Output Dashboard ---
    print("\n" + "="*50)
    print(f" 🦿 [MOTION DETECTED] | Step Number: {step_count}")
    print("="*50)
    print(f" ACTION STATUS      : {movement}")
    print(f" FACING DIRECTION    : Towards {compass_dir} ({heading_deg:.1f}°)")
    print(f" TOTAL DISTANCE WALKED : {total_distance:.2f} units")
    print("-" * 50)

```

```

print(f" ORIGIN DISPLACEMENT RELA : {origin_relation}")
print(f" NET DISTANCE FROM CENTER: {straight_line_dist:.2f} units")
print("-" * 50)
print(f" GRID COORDINATES:")
print(f"  X (East(+)/West(-)) : {x_pos:.2f} units")
print(f"  Y (North(+)/South(-)): {y_pos:.2f} units")
print("="*50)

# --- Engine Start ---
init_lsm303()
print("LSM303 Origin Displacement Tracker Online.")

while True:
    try:
        ax, ay, az, mx, my, mz = read_sensor_data()
        accel_magnitude = math.sqrt(ax**2 + ay**2 + az**2)
        dynamic_force = abs(accel_magnitude - GRAVITY_BASELINE)
        current_time = time.time()

        if dynamic_force > STEP_THRESHOLD and time.time_diff(current_time,
step_cooldown_timer) > 400:
            process_step(mx, my)
            step_cooldown_timer = current_time

        time.sleep(0.02)
    except Exception as e:
        Continue

```